AWS Data Platform DevOps - krótkie przeglądy kodu Python

To zadania ustne. Nie musisz otwierać edytora, pisać całej funkcji, implementować testów ani korzystać z SDK AWS. Przeznacz około czterech minut na zadanie 1, dwóch minut na zadanie 2 i czterech na własne doświadczenia z automatyzacją.

Polska wersja służy przygotowaniu do rozmowy. Zgodnie z ustalonym formatem sama rozmowa i wyjaśnianie toku rozumowania odbywają się po angielsku.

Zadanie 1 - nazwy nieudanych zadań

Poniższe rekordy reprezentują zadania przetwarzające dane:

```
jobs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "FAILED"},
]
```

Każde wywołanie powinno zwracać wyłącznie nazwy nieudanych zadań z tego wywołania, niezależnie od wcześniejszych wywołań. Funkcja zawiera dwa błędy:

```
def failed_jobs(jobs, failed=[]):
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
        return failed
```

Pytania

1. Co funkcja zwróci dla przykładowych danych i dlaczego?
2. Jaka drobna zmiana sprawi, że sprawdzi wszystkie zadania przed zwróceniem wyniku?
3. Po wprowadzeniu tylko tej zmiany, co zwrócą dwa kolejne wywołania `failed_jobs(jobs)` dla przykładowych danych? Przyjmij świeżą definicję funkcji i brak innych wywołań. Wyjaśnij drugi błąd i sposób jego naprawy.
4. Po naprawieniu obu błędów, co powinna zwrócić dla pustych danych oraz gdy wszystkie zadania zakończyły się sukcesem?

Możesz wyjaśnić obie poprawki ustnie; wpisywanie kodu jest opcjonalne.

Przyjmij, że każdy rekord zawiera poprawne pola `name` i `status`. Nie musisz projektować walidacji, ponowień, logowania ani systemu monitorowania dla tego fragmentu.

Zadanie 2 - zadania do ponowienia

Poniższe rekordy reprezentują zakończone uruchomienia przetwarzania:

```
runs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "TIMED_OUT"},
    {"name": "load_prices", "status": "CANCELLED"},
]
```

Do ponowienia należy wybrać wyłącznie zadania ze statusem `FAILED` lub `TIMED_OUT`. Poniższa funkcja nie przestrzega tej reguły:

```
def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] == "FAILED" or "TIMED_OUT":
            retryable.append(job["name"])
    return retryable
```

Pytania

1. Co zwróci `retryable_jobs(runs)`, a co powinna zwrócić?
2. Dlaczego warunek wybiera także zadania udane lub anulowane?
3. Jak poprawisz warunek? Możesz wyjaśnić zmianę ustnie; wpisywanie kodu jest opcjonalne.
4. Jakie proste wejścia sprawdzisz, aby potwierdzić działanie poprawki?

Przyjmij, że każdy rekord zawiera poprawne pola `name` i `status`. Wyjaśnij tylko wybór zadań; mechanizm ponowień, integracja AWS i pisanie testów nie są wymagane.

Krótka rozmowa o doświadczeniu - 4 minuty

Po obu zadaniach opisz jedną automatyzację w Pythonie, nad którą osobiście pracowałeś: co robiła, jak obsługiwała błędy i jak sprawdzałeś ją przed wdrożeniem. W tej części również nie trzeba nic implementować.